BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
Department of Computer Engineering

Name	Arsalan Nabi Ahmed Sayed
UID no.	2023300197
Experiment No.	2

Aim:	To design, implement, train, and evaluate a Generative Image Model — chosen from an Autoencoder, Variational Autoencoder (VAE), Generative Adversarial Network (GAN), Diffusion Model, or Transformer-based Image Generator — on a selected image dataset, and to investigate the effect of at least one architectural or hyperparameter modification on image quality, convergence behavior, and computational performance through comparative analysis.
Experiment 2	
Theory:	An autoencoder learns to compress input data into a smaller latent representation (encoder) and reconstruct it back (decoder), with depth and regularization controlling how much structure and generalization it captures versus just memorizing/copying input. A VAE extends this by encoding inputs as probability distributions instead of fixed points, using a KL-divergence term to shape the latent space into a smooth, standard-normal structure that can be randomly sampled to generate new data. β-VAE adds a weighting factor on that KL term to push the latent dimensions toward capturing independent, disentangled factors of variation, trading off reconstruction accuracy for interpretability and controllability. GANs pit two networks against each other: a Generator maps random noise to images while a Discriminator tries to tell real MNIST images from generated ones, and this adversarial game pushes the Generator toward producing realistic digits with no direct pixel-level supervision. We built three variants — a dense-layer baseline, a class-conditional version (cGAN) that lets us request a specific digit by feeding a label alongside the noise, and a convolutional version (DCGAN) that uses transposed convolutions for sharper, more spatially coherent images — then combined conditioning and convolutions into a Conditional DCGAN. Training used a manual GradientTape loop with separate optimizers for G and D, since Keras's .trainable = False freezing trick is unreliable in recent TF versions. We also applied label smoothing (softening hard 0/1 targets into ranges like 0.8–1.0 and 0.0–0.2) to prevent the Discriminator from becoming overconfident, which stabilizes the adversarial balance and improves image quality.

Q&A:	**Basic** 1. **Why do Autoencoders compress data?** Compression forces the network through a bottleneck (latent space) smaller than the input, so it must learn only the most essential, non-redundant features to reconstruct the input, discarding noise and irrelevant detail. 2. **Autoencoder vs Variational Autoencoder?** A standard Autoencoder learns a fixed deterministic latent vector per input. A VAE learns a probability distribution (mean, variance) over latent space, enabling smooth interpolation and sampling new, valid data points, not just reconstruction. 3. **What is latent space?** A lower-dimensional, learned representation space where compressed data lives. Similar inputs map to nearby points, encoding essential structure/features of the data in a compact form, useful for generation, interpolation, and analysis. 4. **Why is image normalization necessary?** Normalizing pixels (e.g., to [-1,1] or [0,1]) keeps activations and gradients in a stable numeric range, prevents exploding/vanishing gradients, speeds convergence, and matches the generator's output activation (like `tanh`). **Intermediate** 5. **Why does a GAN need both Generator and Discriminator?** The Discriminator provides the training signal — without it, there's no way to judge "realism." Their adversarial competition pushes the Generator to improve, since no fixed loss function alone can define realistic images. 6. **What is mode collapse?** The Generator produces limited, repetitive outputs (e.g., only a few digit styles) instead of the full data diversity, because it found a narrow shortcut that reliably fools the Discriminator without learning the full distribution. 7. **Binary Cross-Entropy vs Mean Squared Error?** BCE measures divergence between predicted and true probabilities, ideal for classification (real/fake). MSE measures squared pixel/value differences, suited for regression/reconstruction tasks. GANs use BCE since D outputs a probability.

8. **Why LeakyReLU over ReLU in GANs?**
   ReLU zeroes all negative inputs, causing "dying neurons" with no gradient flow. LeakyReLU allows a small negative slope, preserving gradient flow through negative activations, which stabilizes training in both G and D.

Advanced

9. **Why are Diffusion Models more stable than GANs?**
   Diffusion models train via a straightforward denoising objective (predict added noise) rather than an adversarial minimax game, avoiding unstable dynamics like mode collapse or vanishing gradients from competing networks.

10. **Explain Fréchet Inception Distance (FID).**
    FID compares statistics (mean, covariance) of real vs generated image features extracted from a pretrained Inception network. Lower FID means generated images are statistically closer to real ones — a common quality/diversity metric.

11. **How does increasing latent dimension affect generated images?**
    More latent dimensions let the generator encode finer, more diverse variation, potentially improving detail and diversity. Too high can add noise/redundancy and complicate training; too low limits expressiveness and causes mode collapse.

12. **Why do GANs generate blurry/distorted images?**
    Causes include insufficient training, an imbalanced G/D (one overpowering the other), mode collapse, small/simple architectures, or dense layers lacking spatial awareness (versus convolutional layers, which preserve local pixel structure better).

Results:

1.Shallow autoencoder

Model: "shallow_autencoder"

Layer (type)	Output Shape	Param #
input_layer_15 (InputLayer)	(None, 784)	0
dense_54 (Dense)	(None, 64)	50,240
dense_55 (Dense)	(None, 784)	50,960

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.1974 - val_loss: 0.1319
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1159 - val_loss: 0.1019
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0956 - val_loss: 0.0883
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0856 - val_loss: 0.0815
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0804 - val_loss: 0.0780
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0778 - val_loss: 0.0761
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0763 - val_loss: 0.0751
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0754 - val_loss: 0.0744
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0749 - val_loss: 0.0739
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0744 - val_loss: 0.0736
```

2.Deep autoencoder

Model: "deep_autoencoder"

Layer (type)	Output Shape	Param #
input_layer_16 (InputLayer)	(None, 784)	0
dense_56 (Dense)	(None, 256)	200,960
dense_57 (Dense)	(None, 128)	32,896
dense_58 (Dense)	(None, 64)	8,256
dense_59 (Dense)	(None, 128)	8,320
dense_60 (Dense)	(None, 256)	33,024
dense_61 (Dense)	(None, 784)	201,488

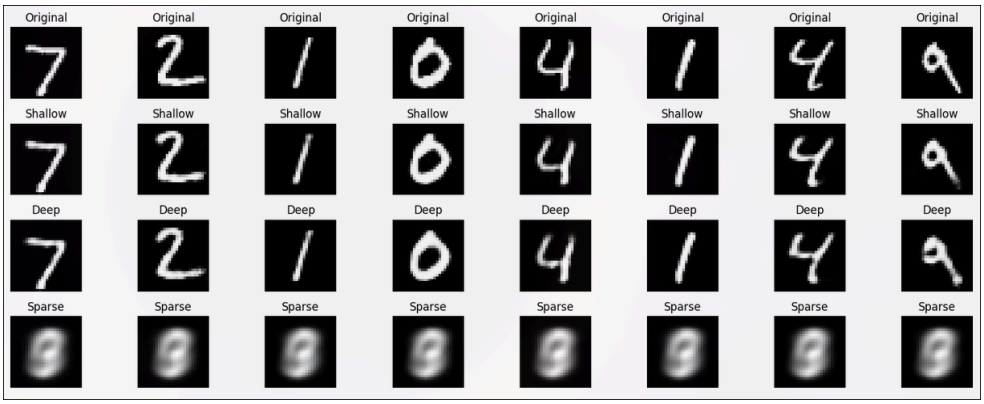
```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 0.1743 - val_loss: 0.1151
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1073 - val_loss: 0.1009
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0971 - val_loss: 0.0925
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0919 - val_loss: 0.0888
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0883 - val_loss: 0.0863
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0859 - val_loss: 0.0840
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0840 - val_loss: 0.0822
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0825 - val_loss: 0.0811
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0813 - val_loss: 0.0799
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0802 - val_loss: 0.0791
```

3.Sparse autoencoder

```
Model: "sparse_autoencoder"
```

Layer (type)	Output Shape	Param #
input_layer_19 (InputLayer)	(None, 784)	0
dense_66 (Dense)	(None, 64)	50,240
dense_67 (Dense)	(None, 784)	50,960

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - loss: 0.6192 - val_loss: 0.5538
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.5054 - val_loss: 0.4641
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4323 - val_loss: 0.4053
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3840 - val_loss: 0.3659
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3512 - val_loss: 0.3388
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3284 - val_loss: 0.3196
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3122 - val_loss: 0.3058
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3004 - val_loss: 0.2957
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2917 - val_loss: 0.2881
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2852 - val_loss: 0.2824
```




```
Latent size 64
Shallow Autoencoder - Test Reconstruction Loss: 0.073649
Deep Autoencoder    - Test Reconstruction Loss: 0.079127
Sparse Autoencoder    - Test Reconstruction Loss: 0.282433
```

4.VAE

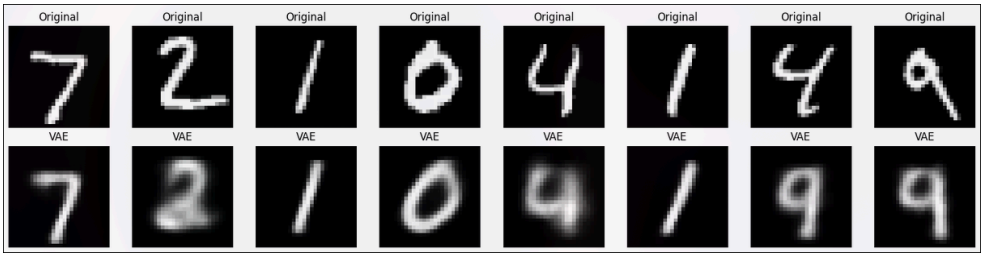
Model: "encoder"

Layer (type)	Output Shape	Param #	Connected to
input_layer_22 (InputLayer)	(None, 784)	0	-
dense_73 (Dense)	(None, 256)	200,960	input_layer_22[0…
dense_74 (Dense)	(None, 128)	32,896	dense_73[0][0]
z_mean (Dense)	(None, 64)	8,256	dense_74[0][0]
z_log_var (Dense)	(None, 64)	8,256	dense_74[0][0]
z (Lambda)	(None, 64)	0	z_mean[0][0], z_log_var[0][0]

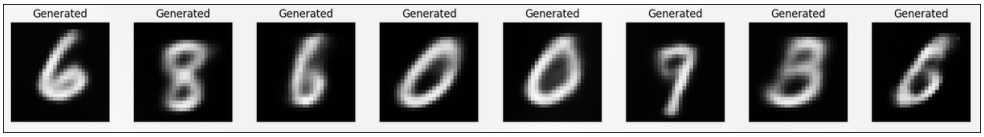
Model: "decoder"

Layer (type)	Output Shape	Param #
input_layer_23 (InputLayer)	(None, 64)	0
dense_75 (Dense)	(None, 128)	8,320
dense_76 (Dense)	(None, 256)	33,024
dense_77 (Dense)	(None, 784)	201,488

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - kl_loss: 0.5935 - loss: 34.5962 - reconstruction_loss: 34.0027
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 1.1374 - loss: 32.4650 - reconstruction_loss: 31.3276
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 2.1060 - loss: 30.9225 - reconstruction_loss: 28.8165
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 3.1250 - loss: 29.5819 - reconstruction_loss: 26.4569
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 3.6504 - loss: 28.8103 - reconstruction_loss: 25.1599
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 4.0015 - loss: 28.3659 - reconstruction_loss: 24.3644
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 4.2797 - loss: 27.9752 - reconstruction_loss: 23.6955
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 4.4341 - loss: 27.7953 - reconstruction_loss: 23.3612
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 4.5298 - loss: 27.6507 - reconstruction_loss: 23.1209
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 4.6136 - loss: 27.5442 - reconstruction_loss: 22.9306
```

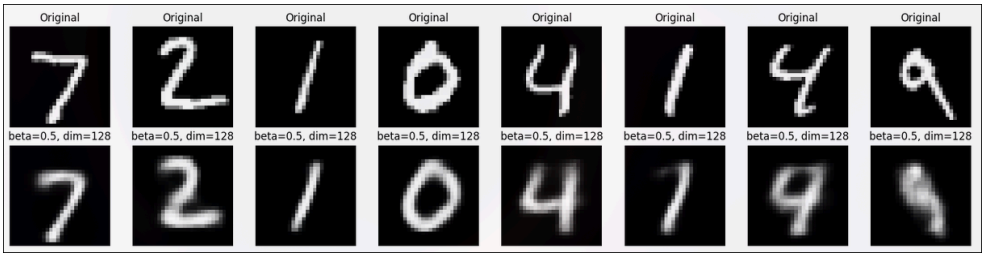


Generation from gaussian noise:

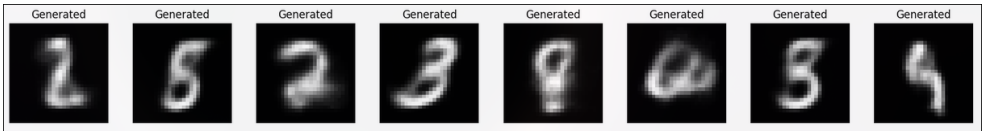


5.Beta VAE

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - kl_loss: 1.3679 - loss: 34.0141 - reconstruction_loss: 33.3301
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 3.8049 - loss: 30.4887 - reconstruction_loss: 28.5863
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 5.8681 - loss: 28.3986 - reconstruction_loss: 25.4645
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 6.7908 - loss: 26.2403 - reconstruction_loss: 22.8449
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 7.1898 - loss: 25.2051 - reconstruction_loss: 21.6102
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 7.4611 - loss: 24.6991 - reconstruction_loss: 20.9685
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 7.7166 - loss: 24.2109 - reconstruction_loss: 20.3526
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 7.9410 - loss: 23.9716 - reconstruction_loss: 20.0011
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 8.1612 - loss: 23.7655 - reconstruction_loss: 19.6849
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - kl_loss: 8.3174 - loss: 23.6029 - reconstruction_loss: 19.4442
```

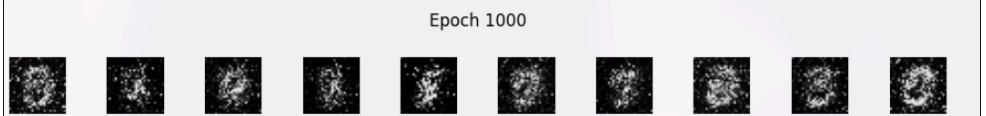


Generation from random noise:

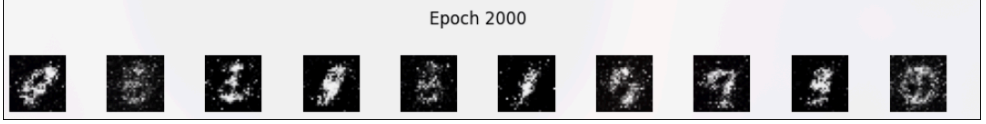


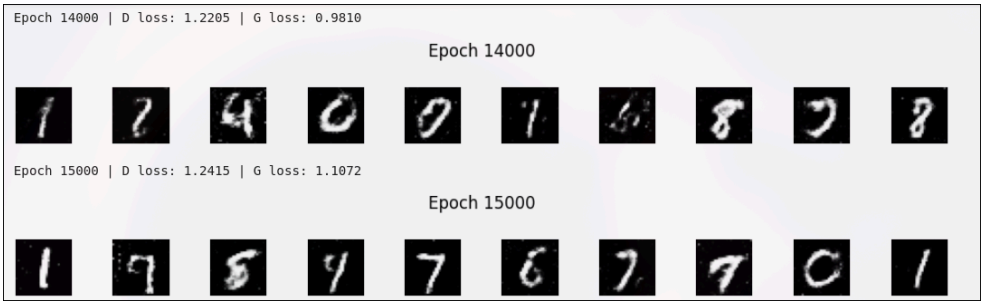
1.Vanilla GAN

```
Epoch 1000 | D loss: 1.2966 | G loss: 0.7284
```

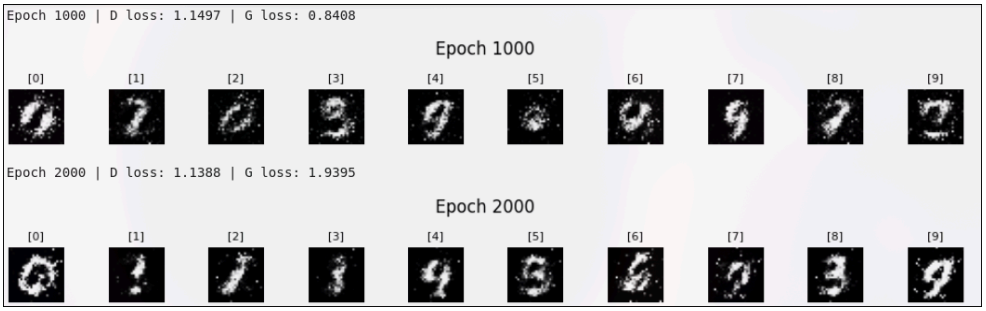


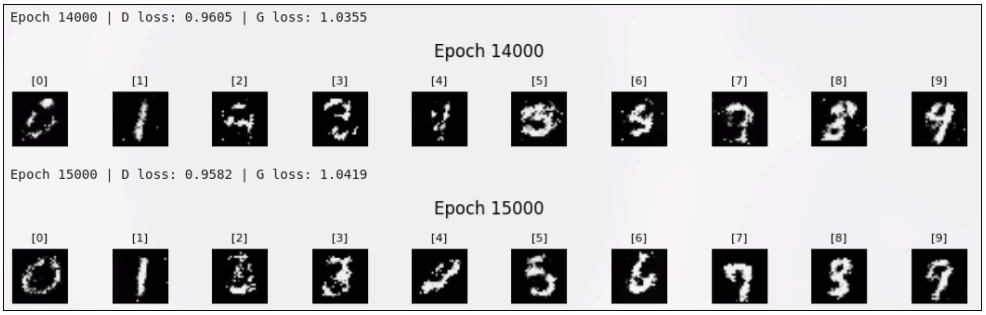

```
Epoch 2000 | D loss: 1.2856 | G loss: 1.0557
```

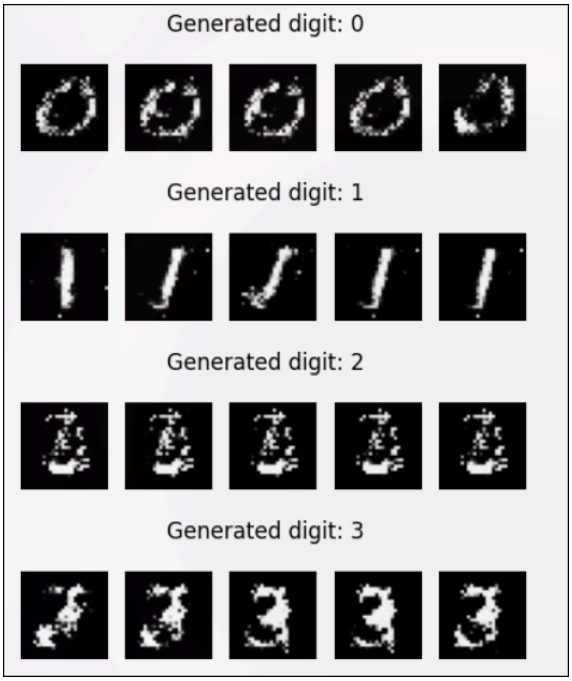




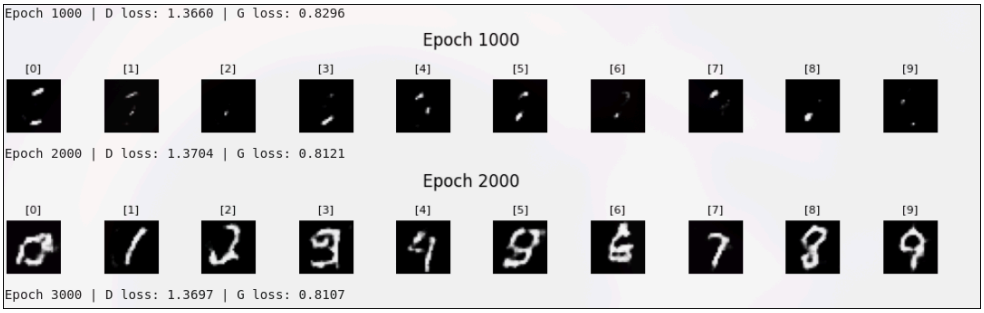
2.Conditional GAN

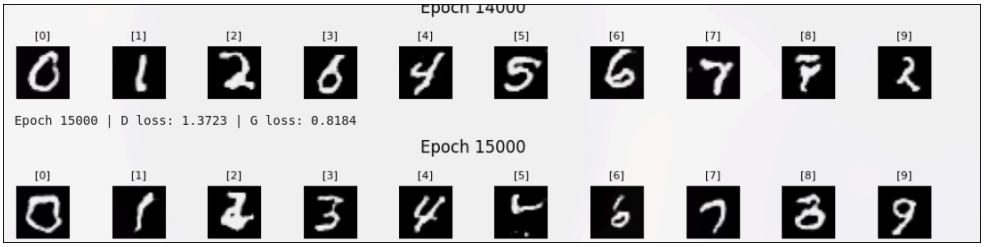


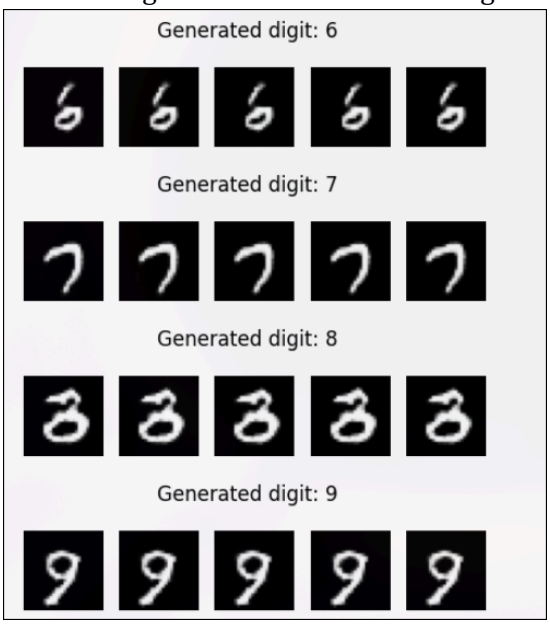




3.Conditional DCGAN





	Generate digit from random noise and given condition: 
Conclusion:	In this experiment, I implemented shallow and deep autoencoders to compress and reconstruct MNIST digits, with the deep version learning richer features through multiple layers. We then explored regularization (L2, dropout, L1 sparsity) to fix issues from an oversized latent space, and a sparse autoencoder that forces only a few latent units active per input for efficient feature learning. Finally, we built VAEs, which encode inputs as distributions instead of fixed points and use a KL-divergence term to organize the latent space, enabling new-image generation. The β-VAE extended this by weighting the KL term to encourage disentangled, interpretable latent factors, at the cost of reconstruction quality and risk of posterior collapse. This experiment implemented and compared three GAN architectures — Baseline, Conditional, and DCGAN — on the MNIST dataset, demonstrating how adversarial training enables a Generator to learn realistic digit distributions purely from a Discriminator's feedback signal. Conditioning on class labels allowed targeted generation of specific digits, while convolutional layers (DCGAN) produced sharper, more spatially coherent images than dense layers alone. Training stability, a recurring challenge, was improved through techniques like manual gradient-tape optimization and label smoothing, highlighting that GAN performance depends as much on training dynamics as on architecture choice.